

Best Practices for Object Diagram

An **Object Diagram** represents instances of classes at a particular moment in time. It's crucial for understanding the state of the system and how objects are related.

1. Keep it Simple and Focused

- **Show only relevant objects:** Avoid overloading the diagram with too many objects. Only include those that are necessary to represent the scenario being modeled.
- **Limit the number of instances:** Focus on a subset of objects that represent a specific scenario or interaction.

2. Use Clear Object Names

- The object name should be clear and descriptive. It should represent the instance and sometimes its state (e.g., `John_Student`).
- Include the object's class name, followed by its current state (e.g., `student1:name="John", grade="A"`).

3. Show Important Relationships

- Represent associations and references clearly by showing relationships between objects (e.g., `Student` has a `Result`).
- Use simple lines to show associations and arrows to represent dependencies.

4. Consistency

- Ensure that object names, attribute values, and the diagram's layout remain consistent with other parts of your design.
- When using attributes in object instances, use consistent formats, such as showing the values in a specific format (`<attributeName>:<value>`).

5. Avoid Redundancy

- Don't repeat information that is already present in the Class Diagram unless necessary for the scenario.
- Object Diagrams should only depict the state of objects, not duplicate the class-level design.

Best Practices for Class Diagram

A **Class Diagram** provides a static view of the system's structure, representing classes, attributes, methods, and relationships.

1. Keep it Simple and Abstract

- Focus on **high-level** classes and avoid unnecessary details. Only include the attributes and methods that are essential for understanding the system's structure.
- Avoid overcomplicating with too many classes, especially in the initial stages of design.

2. Use Meaningful Names for Classes and Attributes

- **Class names** should be nouns that clearly describe the object or concept (e.g., `Student`, `Course`, `Result`).
- **Attribute names** should describe the characteristics of the object (e.g., `studentId`, `email`, `grade`).
- **Method names** should represent actions (e.g., `enrollInCourse()`, `assignGrade()`).

3. Define Relationships Clearly

- Clearly define the types of relationships between classes using appropriate UML notations:
 - **Association**: Represented by a simple line, indicating that classes are related.
 - **Inheritance**: Represented by a line with a triangle, indicating a superclass/subclass relationship.
 - **Aggregation/Composition**: Represented by lines with diamonds, denoting "whole-part" relationships (composition has a stronger relationship than aggregation).
- Use the right multiplicity (e.g., one-to-many, many-to-many) to describe how classes are related.

4. Show Interfaces and Abstract Classes When Needed

- If you're modeling interfaces, use the dashed line with a triangle pointing to the implementing class. This clarifies which class is fulfilling a contract.
- Abstract classes should be represented with italics or a clear indication that they cannot be instantiated.

5. Use Proper Access Modifiers

- Indicate whether attributes and methods are **public**, **private**, or **protected** (e.g., `+`, `-`, `#`).
- This helps clarify the visibility and encapsulation of each component in the class.

6. Group Classes into Packages

- In larger systems, group related classes into packages. This reduces clutter and improves readability.
- Use **packages** to logically group related classes (e.g., `student`, `course`, `results`).

Best Practices for Sequence Diagram

A **Sequence Diagram** models the interaction between objects over time, focusing on the sequence of messages.

1. Clear and Consistent Object Naming

- The objects in the diagram should be clearly labeled with meaningful names. Use class names followed by object identifiers (e.g., `Student1`, `Teacher_MrSmith`).
- Use consistent naming conventions for messages, such as `placeOrder()`, `enrollInCourse()`.

2. Limit the Number of Objects in the Diagram

- Too many objects can make the sequence diagram cluttered and hard to read. Limit the number of objects to only those essential for the particular scenario you're modeling.
- If necessary, break complex interactions into smaller diagrams.

3. Represent Lifelines and Activations Properly

- **Lifelines** represent the existence of objects and are drawn as dashed vertical lines.
- **Activation bars** represent when an object is active and performing a task. Ensure that the lifeline's activation bar is clearly defined for each method call.

4. Use Clear and Meaningful Messages

- Messages should clearly indicate what is happening between the objects. Use consistent naming conventions for method calls (e.g., `getStudentResult()` or `calculateGrade()`).
- Return messages should be dashed arrows, indicating that the method has completed and returned control or a value to the calling object.

5. Ensure Proper Message Ordering

- Messages in a sequence diagram should be drawn in **top-to-bottom** order to reflect the logical sequence of operations. The first message should appear at the top, followed by the subsequent messages.

- Use arrows to indicate the flow of communication, with clear labels for each message.

6. Show Conditionals and Loops When Needed

- If the flow depends on certain conditions, use **alt** (alternatives) or **opt** (optional) boxes to represent decision points or optional operations.
- For loops or repeated actions, use **loop** boxes and clearly show the repetition.

7. Keep It Focused on a Single Use Case

- Each sequence diagram should represent a single use case or scenario, making it easier to follow.
- Avoid combining multiple interactions in one sequence diagram. If a scenario has multiple branches or steps, create separate diagrams for each.

Sample Problem 1: School Results Application

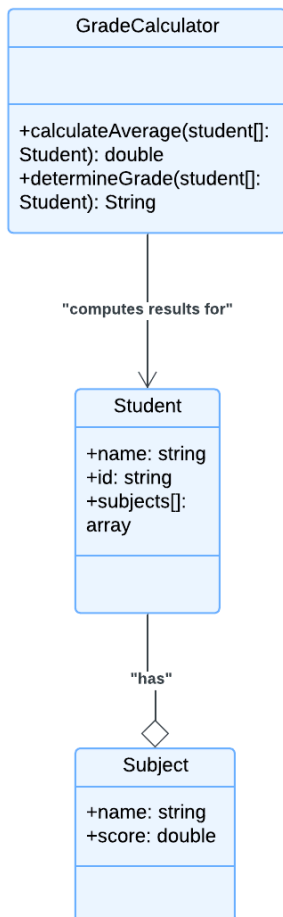
Class Diagram

The class diagram represents the structure of a school results application where students have subjects, and their scores are calculated for grades.

Diagram Description:

- **Classes:** `Student`, `Subject`, `GradeCalculator`
- **Relationships:**
 - A `Student` has multiple `Subject` entries (Aggregation).
 - `GradeCalculator` computes the results for a `Student`.

→ Draw the Class Diagram



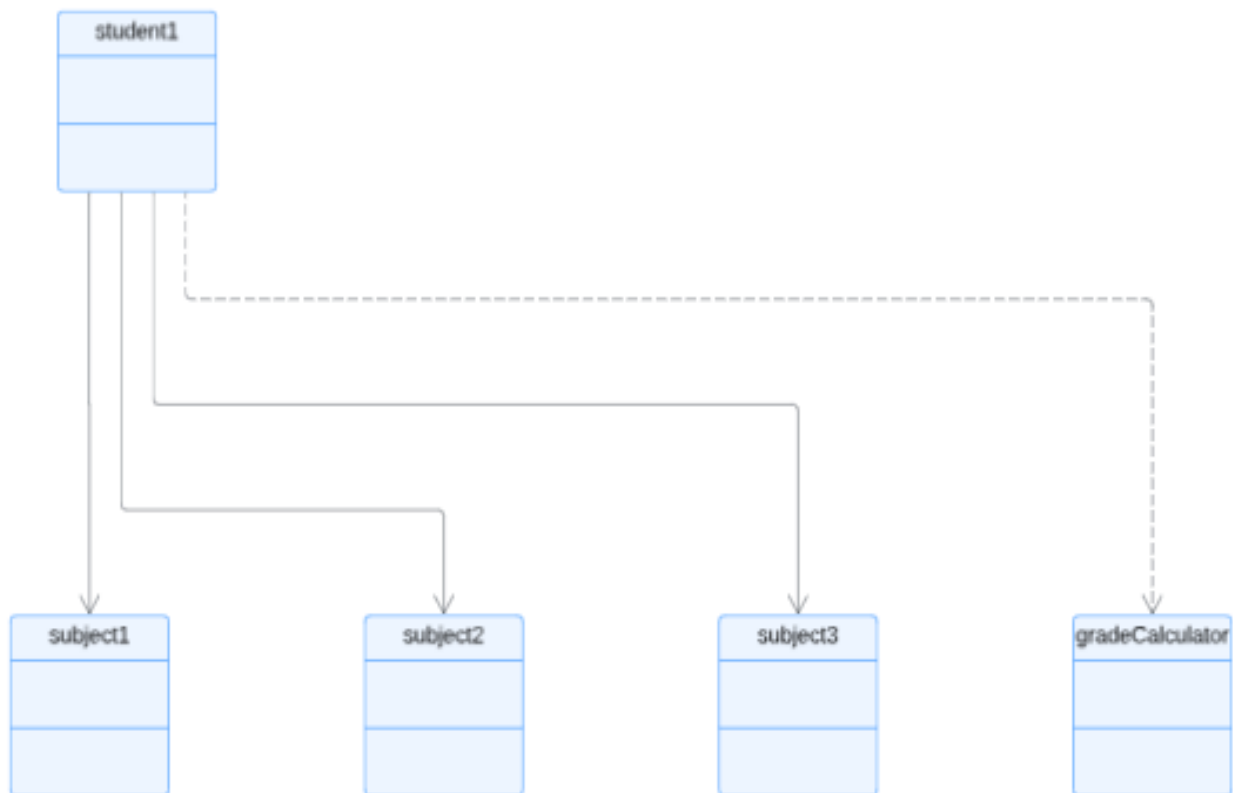
Object Diagram

An object diagram provides a snapshot of the **Student** and their **Subject** objects at a particular point.

Example:

- **Student:** John
- **Subjects:** Maths, Science
- **Marks:** 90, 85

→ Draw the Object Diagram



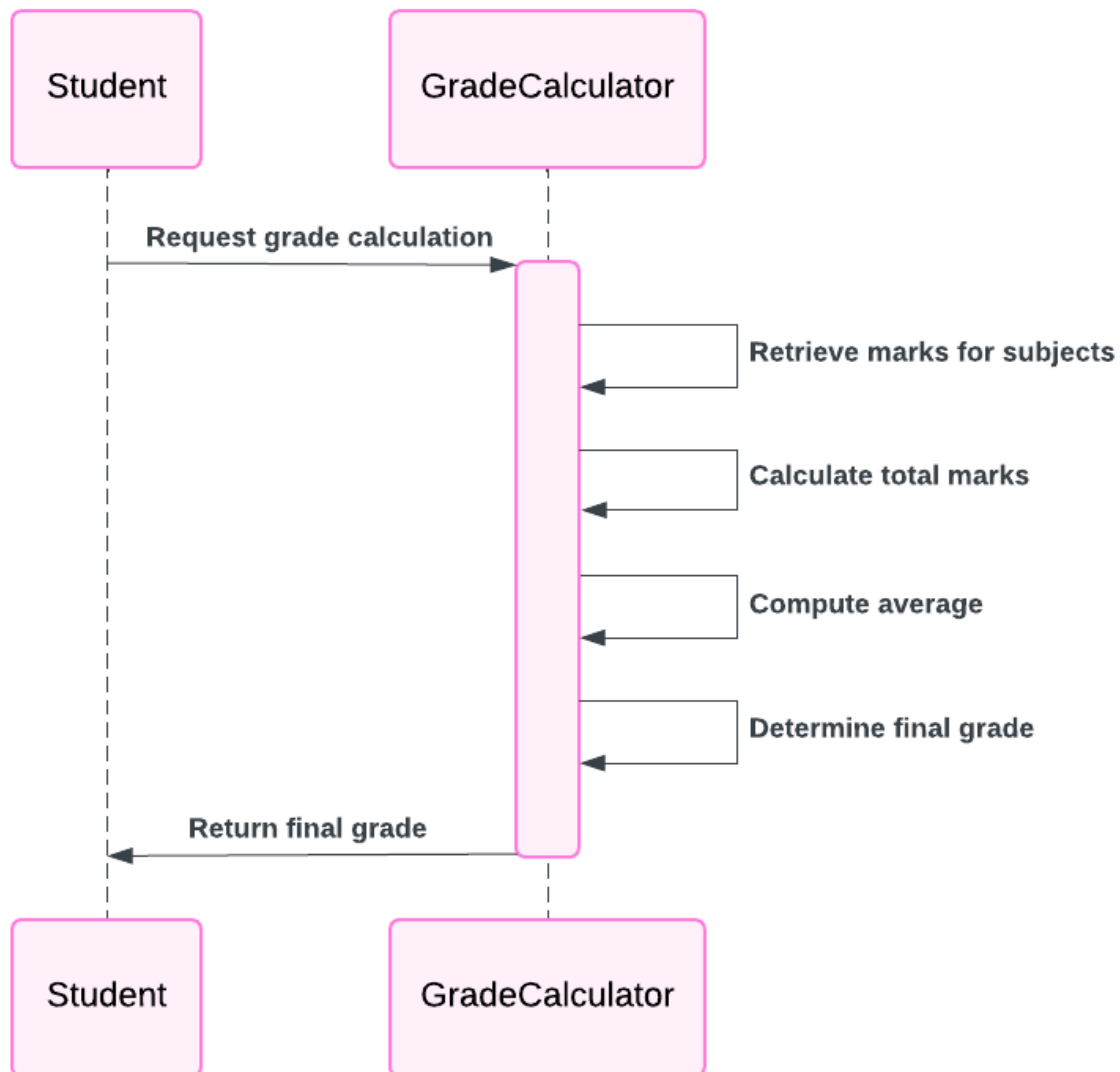
Sequence Diagram

The sequence diagram shows how objects interact to calculate grades.

Scenario: A student requests their grade based on marks in subjects.

Actors:

1. Student
2. GradeCalculator



```
import java.util.Scanner;
```

```
class Student {
    private String name;
    private final String studentId;
    private final Subject[] subjects;

    //constructor to initialize variables
    public Student(String name, String studentId,
Subject[] subjects){
        this.name = name;
        this.studentId = studentId;
        this.subjects = subjects;
    }
    //method to print details
    public void display() {
        System.out.println("\nResult of Student
.....");
        System.out.println("Name : " + this.name);
        System.out.println("Student ID : " +
this.studentId);

        //printing scores of each subject
        for(int i=0; i<subjects.length; i++) {
            System.out.println("Marks of subject " +
subjects[i].name + " is " + subjects[i].score);
        }
        System.out.println("Average of scores : " +
GradeCalculator.calculateAverage(subjects));
        System.out.println("Grade is : " +
GradeCalculator.determineGrade(subjects));
    }
}
```



```
}  
//subject class  
class Subject {  
    String name;  
    double score;  
  
    //constructor to initialize variables  
    public Subject(String name, double score) {  
        this.name = name;  
        this.score = score;  
    }  
}  
//GradeCalculator class  
class GradeCalculator {  
  
    //method to calculate average  
    public static double calculateAverage(Subject[]  
subjects) {  
        //calculating sum of score of students  
        double sum = 0;  
        for(int i=0; i<subjects.length; i++) {  
            sum += subjects[i].score;  
        }  
  
        //calculating the average  
        double avg = sum/(subjects.length);  
  
        //returning the average  
        return avg;  
    }  
}
```

```
//method to determine grades
public static String determineGrade(Subject[]
subjects) {

    //calculating average for subjects
    double avg = calculateAverage(subjects);

    //determining grades
    String grade;
    if(avg > 85) {
        grade = "A+";
    } else if(avg > 75) {
        grade = "A";
    } else if(avg > 60) {
        grade = "B";
    } else if(avg > 45) {
        grade = "C";
    } else if(avg > 33) {
        grade = "D";
    } else {
        grade = "FAIL";
    }
    //returning grade
    return grade;
}

}

public class SchoolResultApplication {
    public static void main(String[] args) {
        //creating Scanner class object
        Scanner input = new Scanner(System.in);
```

```
//taking input for student details
System.out.println("\nFetching details for a
student.... ");
System.out.print("Enter Name : ");
String name = input.nextLine();
System.out.print("Enter student id : ");
String studentId = input.nextLine();
System.out.print("Enter number of subjects : ");
int numberOfSubjects = input.nextInt();

//creating array subjects to store subjects and
a variable to indicate index of array
Subject[] subjects = new
Subject[numberOfSubjects];
int index = 0;

//taking input for subject details of current
student
for (int i = 0; i < numberOfSubjects; i++) {
    System.out.print("Enter name of subject " +
(i+1) + " : ");

    String SubName = input.next();
    System.out.print("Enter score : ");
    double score = input.nextDouble();

    //creating object of Subject class
    Subject subject = new Subject(SubName,
score);
```

```
        //appending the new subject in Subject array
        subjects[index] = subject;
        index++;
    }
    //creating object of Student class
    Student student1 = new Student(name, studentId,
subjects);

    //printing details
    // calculation of average and score in display()
method of student class
    student1.display();

    //closing the input
    input.close();
}
}
```

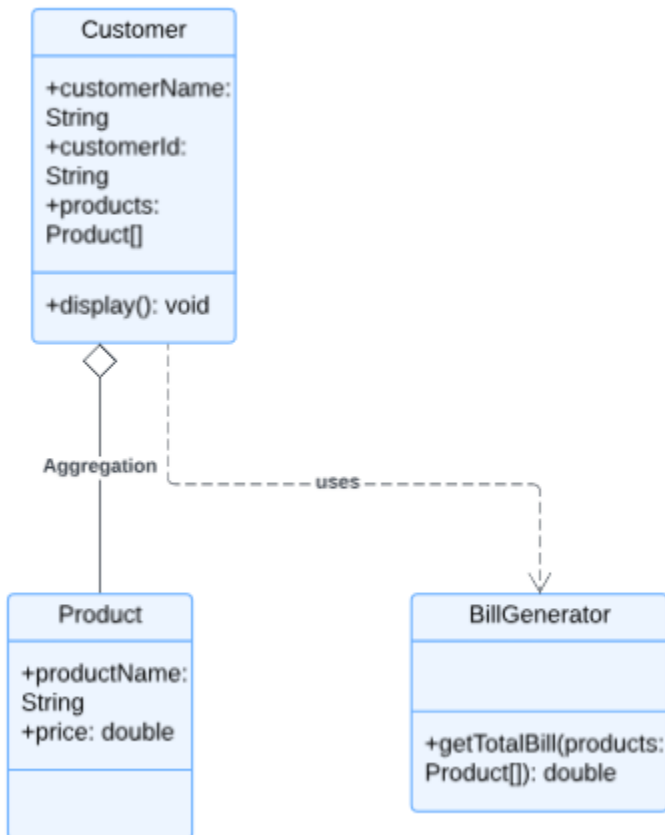
Sample Problem 2: Grocery Store Bill Generation Application

Class Diagram

The class diagram models the system where a customer buys products, and the bill is generated.

Diagram Description:

- **Classes:** Customer, Product, BillGenerator
- **Relationships:**
 - A Customer can purchase multiple Product items (Composition).
 - BillGenerator computes the total for the Customer.



Object Diagram

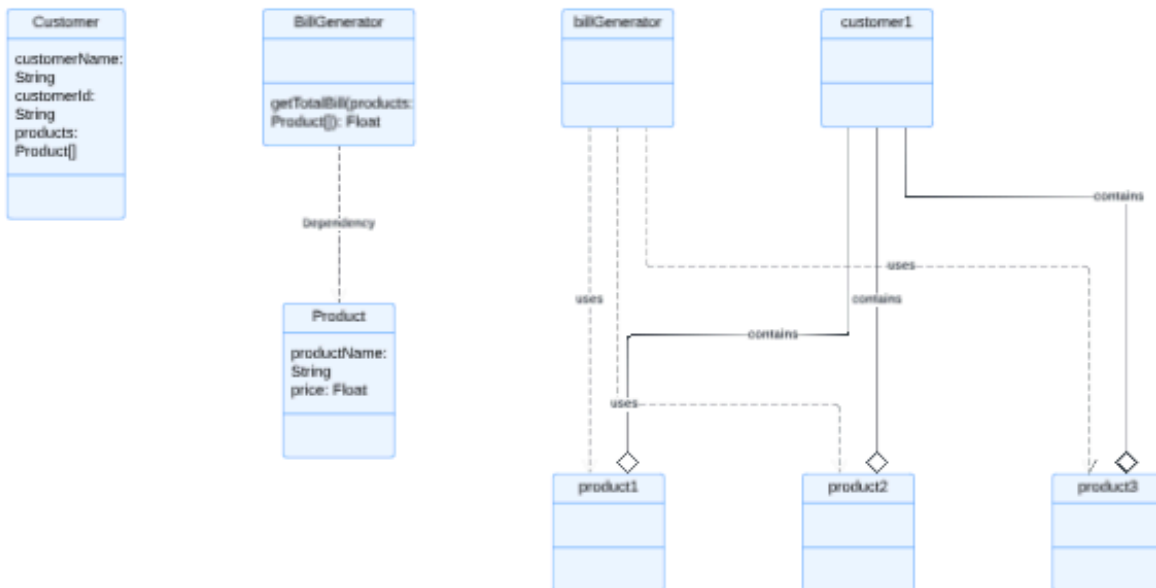
An object diagram shows the details of a Customer and the Product objects they have purchased.

Example:

- **Customer:** Alice

- **Products:**

- Apples (2 kg at \$3 per kg)
- Milk (1 liter at \$2 per liter)



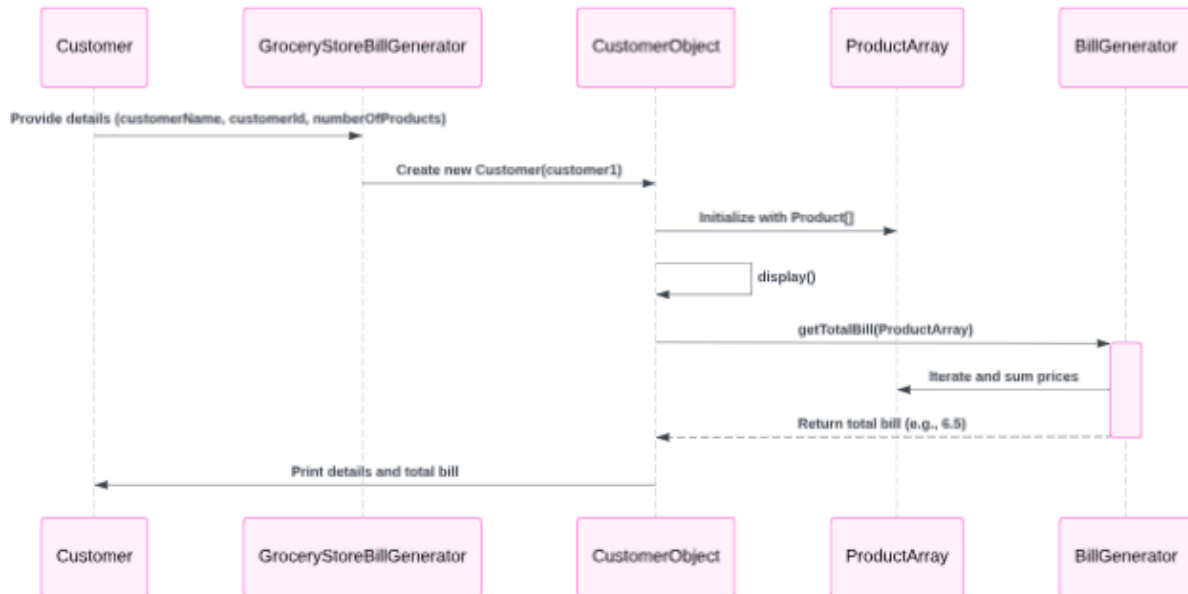
Sequence Diagram

The sequence diagram shows the process of bill generation for a customer.

Scenario: A customer checks out at the grocery store, and the total bill is generated.

Actors:

1. Customer
2. BillGenerator



→ Draw the Sequence Diagram

```

import java.util.Scanner;

//class Customer
class Customer {
    private String customerName;
    private final String customerId;
    private Product[] products;

    //constructor to initialize variables
    public Customer(String name, String customerId,
Product[] products) {
        this.customerName = name;
        this.customerId = customerId;
        this.products = products;
    }
}
    
```

```
//printing details
public void display() {
    System.out.println("\nDetails of shopping
.....");
    System.out.println("Name : " +
this.customerName);
    System.out.println("Customer Id : " +
this.customerId);

    //printing price of each product
    for (int i = 0; i < products.length; i++) {
        System.out.println("Price of product " +
products[i].productName + " is " + products[i].price);
    }
    System.out.println("total Bill : " +
BillGenerator.getTotalBill(products));
}
}
//class Product
class Product {
    String productName;
    double price;

    //constructor to initialize values
    public Product(String productName, double price) {
        this.productName = productName;
        this.price = price;
    }
}
//class BillGenerator
```



```
class BillGenerator {
    //creating the static method to generate total bill
    public static double getTotalBill(Product[]
products) {

        //creating variable total price to indicate
total price
        double totalBill =0;

        //calculating total price
        for(int i=0; i< products.length; i++) {
            totalBill += products[i].price;
        }
        //returning the total price
        return totalBill;
    }
}

public class GroceryStoreBillGenerator {
    public static void main(String[] args) {
        //creating Scanner class object
        Scanner input = new Scanner(System.in);

        //taking input for student details
        System.out.println("\nFetching details for a
customer.... ");
        System.out.print("Enter Name : ");
        String customerName = input.nextLine();
        System.out.print("Enter customer id : ");
        String customerId = input.nextLine();
        System.out.print("Enter number of products : ");
```

```
int numberOfProducts = input.nextInt();

//creating array products to store products and
a variable to indicate index of array
Product[] products = new
Product[numberOfProducts];
int index = 0;

//taking input for product details of current
customer
for (int i = 0; i < numberOfProducts; i++) {
    System.out.print("Enter name of product " +
(i+1) + " : ");
    String productName = input.next();
    System.out.print("Enter price : ");
    double price = input.nextDouble();

    //creating object of Product class
    Product product = new Product(productName,
price);

    //appending the new product in Product array
    products[index] = product;
    index++;
}
//creating object of Customer class
Customer customer1 = new Customer(customerName,
customerId, products);

//printing details
```

```
// calculation of total bill in display()
method of Customer class
customer1.display();

//closing the input
input.close();
}
}
```

Comparison of the Two Scenarios

Feature	School Results Application	Grocery Store Bill Application
Classes	Student, Subject, GradeCalculator	Customer, Product, BillGenerator
Relationships	Aggregation	Composition
Primary Functionality	Calculate grade	Generate total bill
Key Entities	Students, Subjects, Grades	Customers, Products, Bills